King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 10
B Trees

Term 261

Reading Assignment

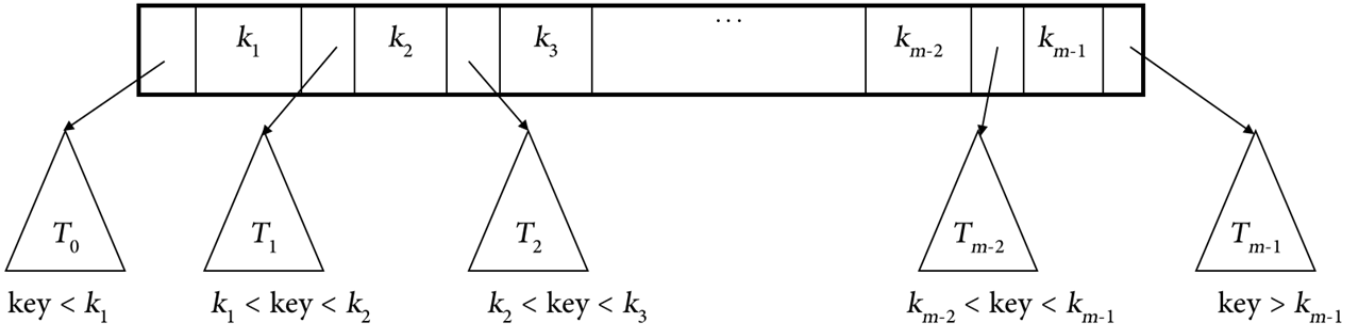
- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-981-4392-78-5
  - Chapter 7 Section 7.1 (7.1.1 and 7.1.3 only)

Objectives

- Discuss the following topics:
  - Multi-Way Trees
- B-Trees
  - What is a B-tree?
  - Why B-trees?
  - Searching a B-tree
  - Insertion in a B-tree
  - Deletion in a B-tree
- NOTE: SOME EXAMPLES IN THIS UNIT ARE ADOPTED FROM INTERNET SOURCES

What is a Multi-way tree?

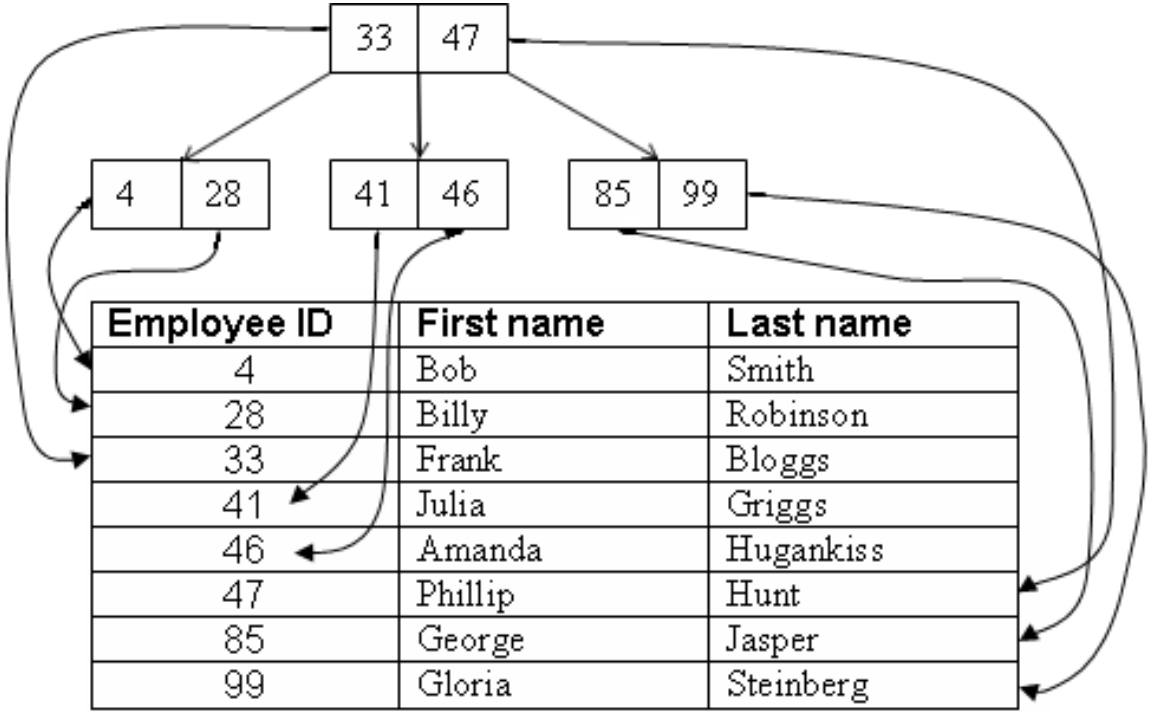
- A multi-way search tree of order m is a tree in which:
  - Each node has m-1 distinct keys and m children; a child may be empty.
  - The keys in each node are sorted.
  - A non-leaf node with n keys contains n + 1 non-empty subtrees.
  - The leaf nodes need not be at the same level.



- The keys and subtrees of a non-leaf node are ordered as: $T_0, k_1, T_1, k_2, T_2, \ldots, k_{m-1}, T_{m-1}$ such that:
  - All keys in subtree T_0 are less than k_1.
  - All keys in subtree T_i , $1 \leq i \leq m-2$, are greater than k_i but less than k_{i+1}.
  - All keys in subtree T_{m-1} are greater than k_{m-1}.

The node structure of a Multi-way tree

- Corresponding to each key there is a data reference that refers to the data record for that key in secondary memory.
- In our representation, we will omit the data references.



Employee ID	First name	Last name
4	Bob	Smith
28	Billy	Robinson
33	Frank	Bloggs
41	Julia	Griggs
46	Amanda	Hugankiss
47	Phillip	Hunt
85	George	Jasper
99	Gloria	Steinberg

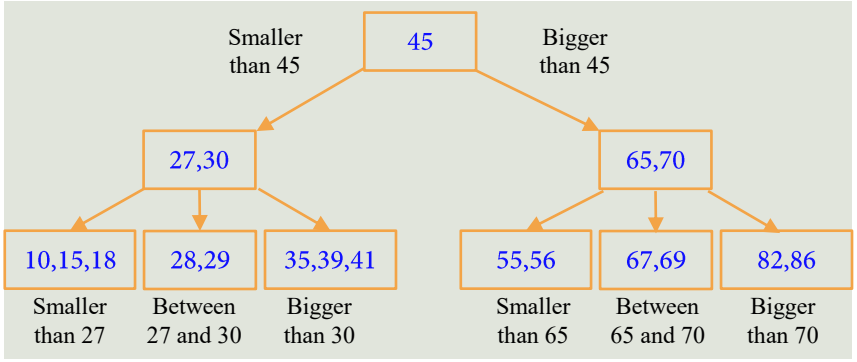
What is a B-Tree?

Nodes are kept at least half full

- A B-tree of order m (or branching factor m), where m > 2, is a self-balancing multiway search tree with the following properties:

	Root node	Non-root node	
Minimum number of keys	1	$\lceil \frac{m}{2} \rceil - 1$	The root has at least 2 children, if it is not a leaf
Minimum number of children (degree)	2	$\lceil \frac{m}{2} \rceil$	Every non-leaf node has at least $\lceil \frac{m}{2} \rceil$ children

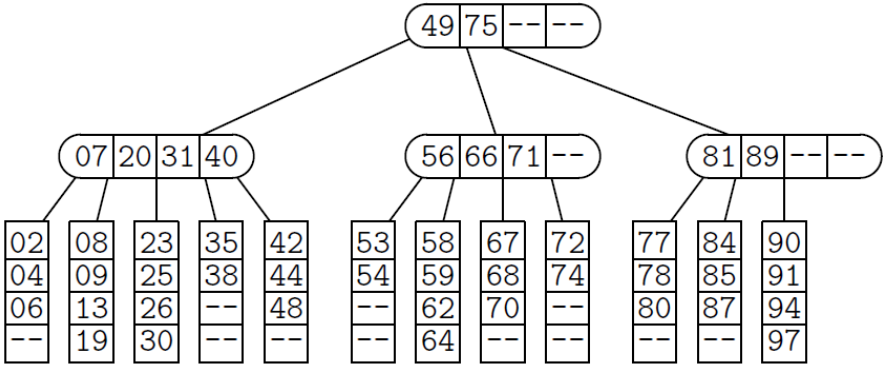
  - Maximum number of keys is m-1; #keys = #children – 1
  - All leaves appear at the same level. So, search, insert, delete all take **O(log$_m$ n)** time.
- Example: A B-tree of order 4



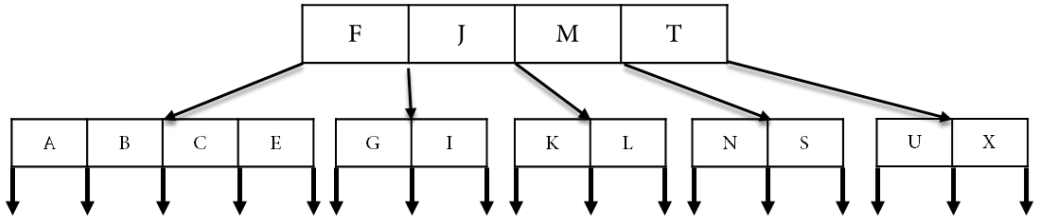
	Non-root node
Min # keys	$\lceil \frac{m}{2} \rceil$ - 1 = 1
Min # non-empty subtrees	
Max # keys	m – 1 = 3
Max # non-empty subtrees	

B-Tree Examples

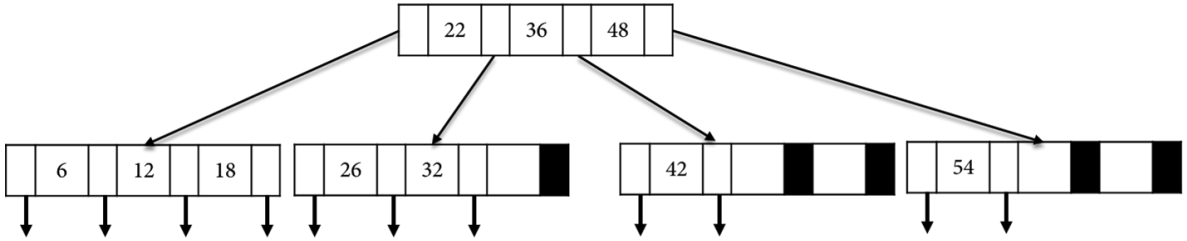
- B-trees of order 5



	Non-root node
Min # keys	$\lceil \frac{m}{2} \rceil - 1 = 2$
Max # keys	$m - 1 = 4$



- A B-tree of order 4



	Non-root node
Min # keys	$\lceil \frac{m}{2} \rceil - 1 = 1$
Max # keys	$m - 1 = 3$

Motivation for Multi-Way Trees

- In memory, AVL trees give fast search $O(\log n)$
- But when data is too large to fit in the memory, we must store it on **disk**
- Disk access is extremely expensive—about 200,000 CPU instructions
  - Minimizing disk accesses becomes crucial.
- Problems with Traditional Trees:
  - Height of the tree: An AVL tree with millions of nodes can be very tall.
  - Searching takes many disk reads
  - Each node in an AVL tree is small, but disks read in **blocks/pages**
- B-Trees were designed to solve these problems:
  - Minimize disk I/O
    - Shallow height: With a large branching factor m
    - Because nodes can have many children, the tree height is much smaller than an AVL tree.
    - ⟶ Fewer levels ⟶ fewer disk accesses.
    - Each node of a B-Tree holds many keys (not just one), filling an entire disk block.
    - ⟶ One disk read retrieves many keys at once.
  - Balanced structure
    - B-Trees are always balanced: all leaf nodes are at the same depth.
    - ⟶ Predictable performance for search, insert, delete.

Comparing B-Trees with AVL Trees

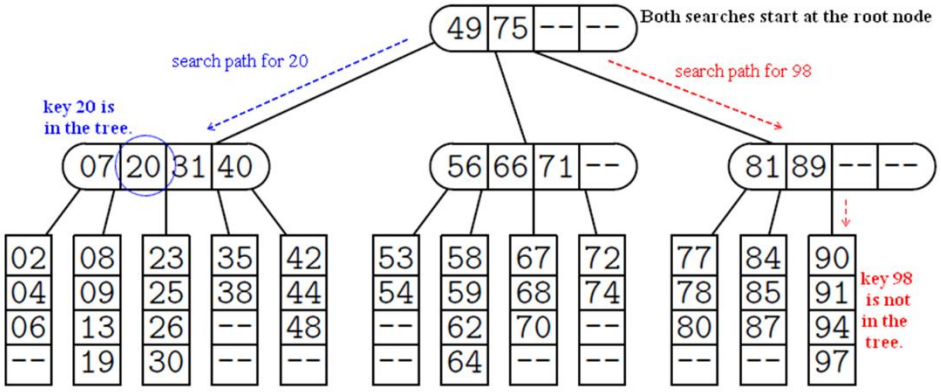
- Let us assume we have **16,000,000 keys** stored on disk. We want to know how many disk accesses are needed to retrieve one key using:
  - a **B-tree**, and
  - an **AVL tree**

B-Tree of order m	AVL Tree with n keys
For a B-tree of order m = 300, the height satisfies: $h <= 1 + \log_{\lceil \frac{m}{2} \rceil} \left(\frac{n+1}{2} \right)$	**The average number of comparisons for an AVL is** $\log n + 0.25$.
With n = 16,000,000, $h \approx 4$.	With n = 16,000,000, $\log n + 0.25 = 24$
Thus, retrieving a key requires only 4 disk accesses in the worst case (assuming the root node is always in main memory).	The average number of comparisons is 24

- Note: As m increases the amount of computation at each node increases; however, this cost is negligible compared to hard-drive accesses
- The most common data structure used for database indices is the B+-tree, which is a variant of the B-tree

Searching a B-Tree

- Searching for KEY:
- Example:
  - Search for 20
  - Search for 98
  - Search for 70




```
BTree-Search(x, k):  // x is a node
   i ⟵ 0
   while i < x.n and k > x.key[i]:
      i ⟵ i + 1
   if i < x.n and k == x.key[i]:
      return (x, i)
   else if x.isLeaf:
      return NOT-FOUND
   else:
      return BTree-Search(x.child[i], k)
```

Non-leaf node

Search KEY among the keys in that node: linear search or binary search.

If found, return the corresponding record

If not found

If KEY < smallest key, follow the leftmost child reference down

If KEY > largest key, follow the rightmost child reference down

If K_i < KEY < K_j, follow the child reference between K_i and K_j

Leaf node

Search KEY among the keys stored in that leaf: linear search or binary search

If found, return the corresponding record. otherwise report not found

Insertion in B-Trees

- Insertion algorithm: Insertion of a key always starts at a leaf node

  If the key to be inserted already exists in the tree, report an error

  If a node overflows, split it into two, propagate the "middle" key to the parent of the node

  If the parent overflows the process propagates upward.

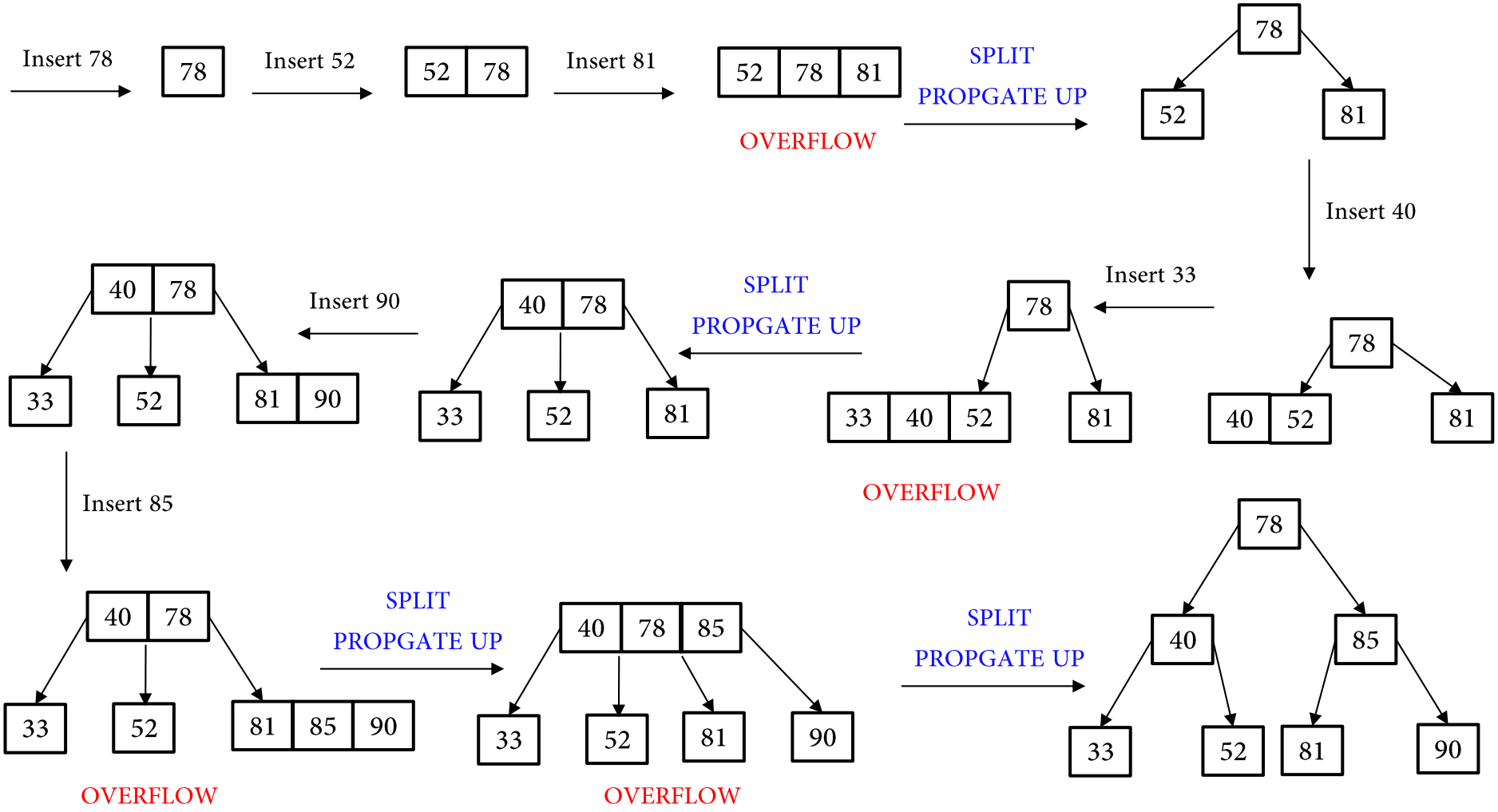
  If the node has no parent, create a new root node.

- Note:
  - A key is inserted with its associated data and data reference.

- OVERFLOW CONDITION:
  - A node of order m overflows if, after a key insertion, it contains m keys.

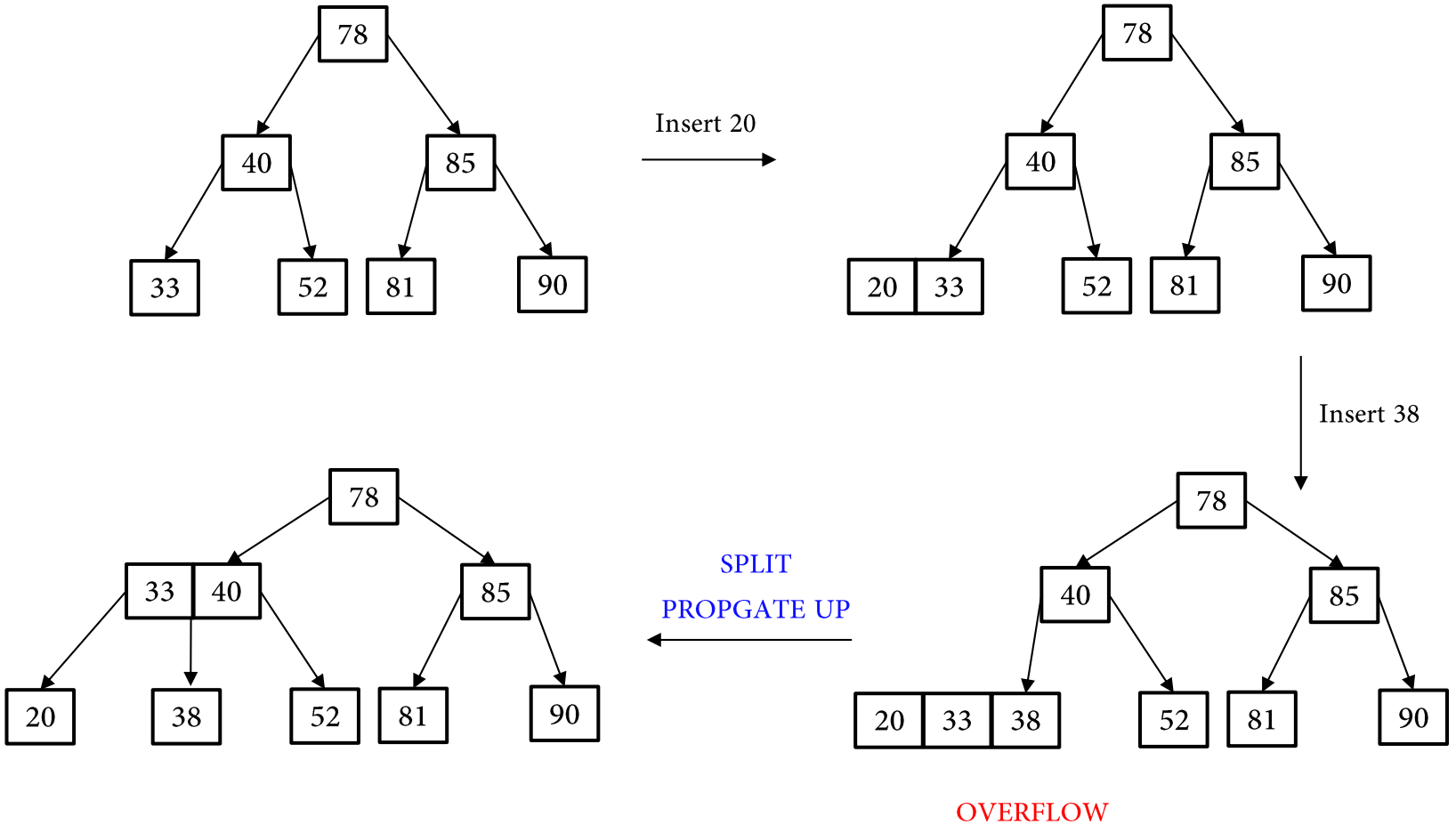
Insertion in B-Trees

- Example: Insert the keys 78, 52, 81, 40, 33, 90, 85, 20, and 38 in this order in an initially empty B-tree of order 3



Insertion in B-Trees

- Example: Insert the keys 78, 52, 81, 40, 33, 90, 85, 20, and 38 in this order in an initially empty B-tree of order 3



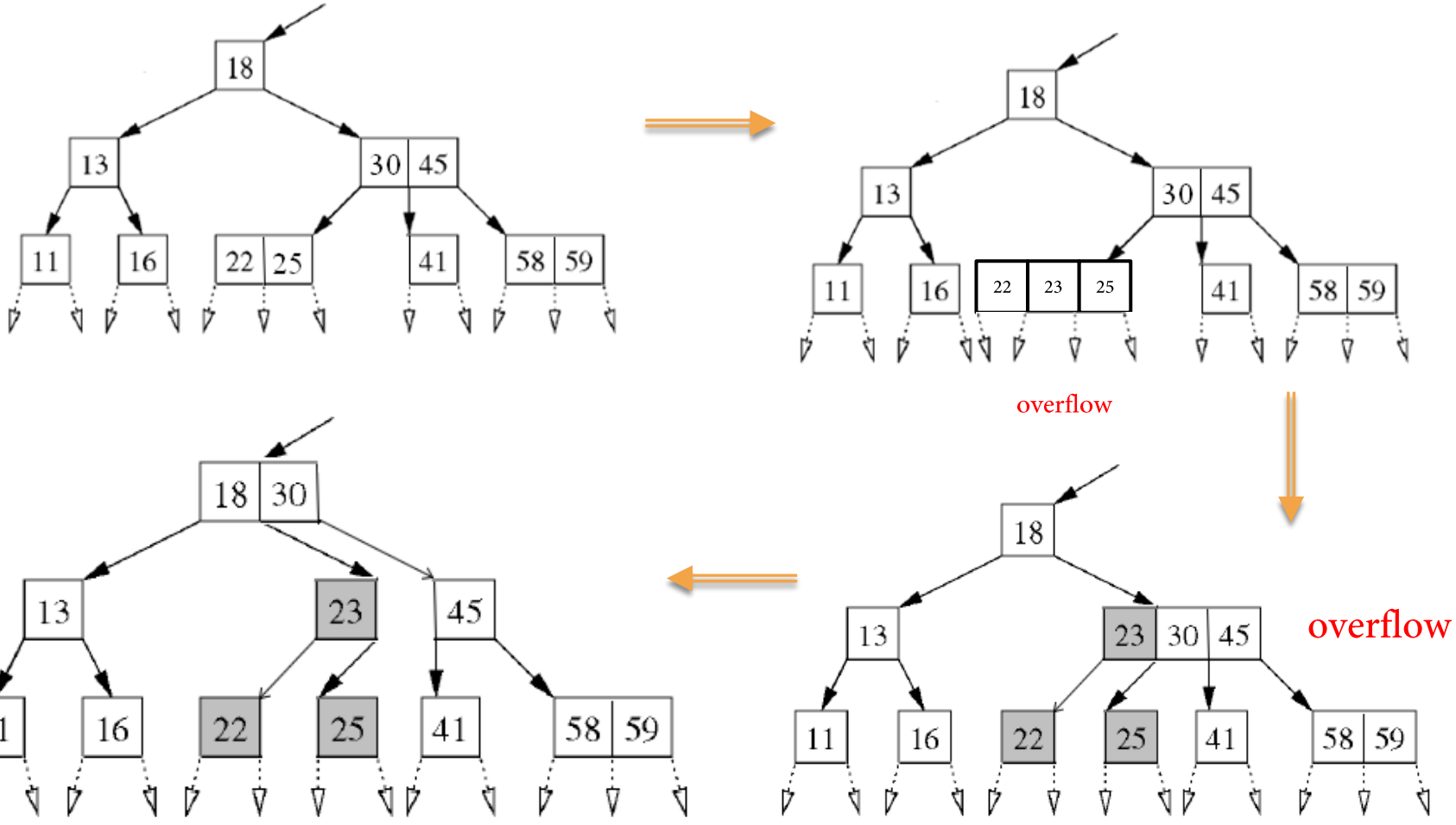
Insertion in B-Trees

- Insertion in a B-tree of odd order
  - Insert 23 in a B-tree of order 3

Max #Keys = m -1 = 3 - 1 = 2

Overflow condition:

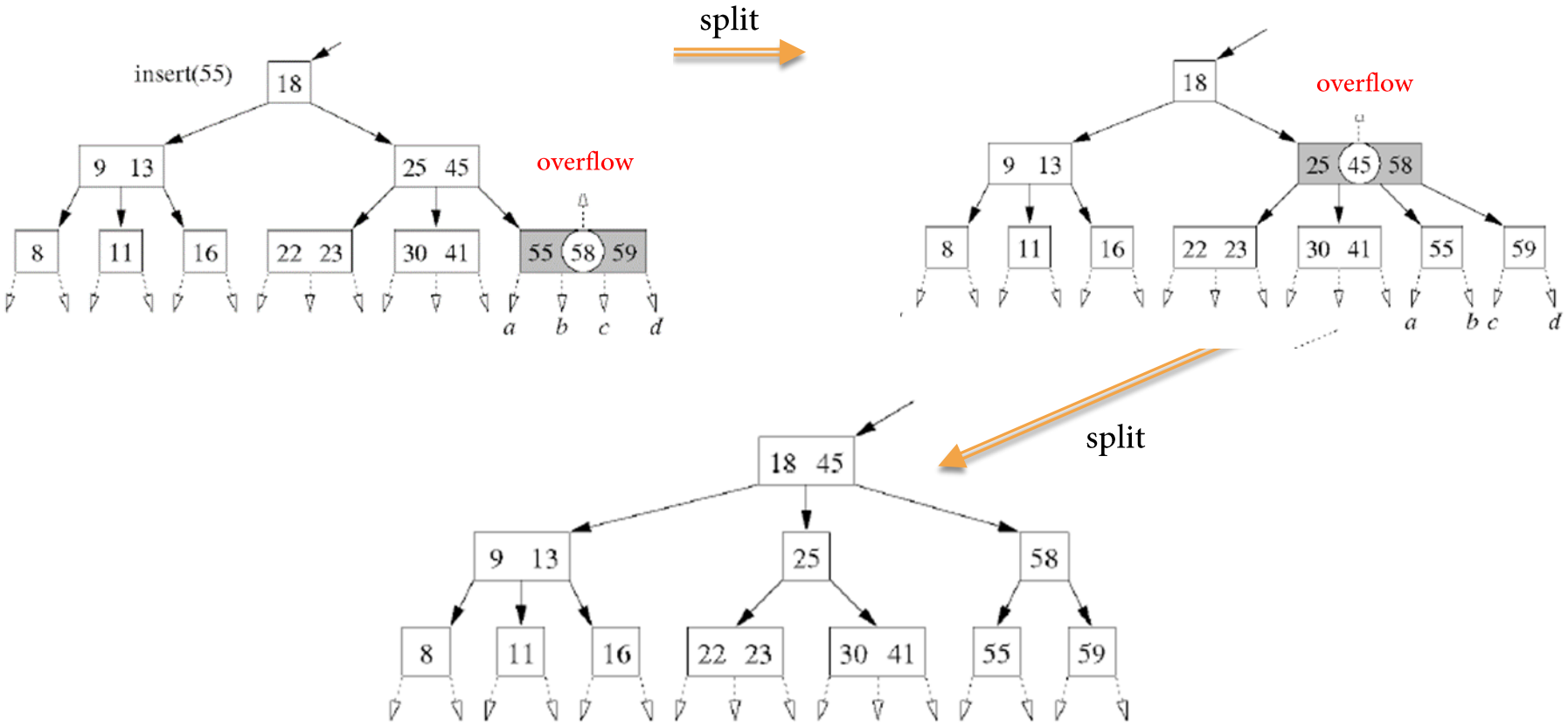
numKeysInNode = 3



Insertion in B-Trees

- Insertion in a B-tree of odd order
  - Insertion of key 55 in a B-tree of order 3

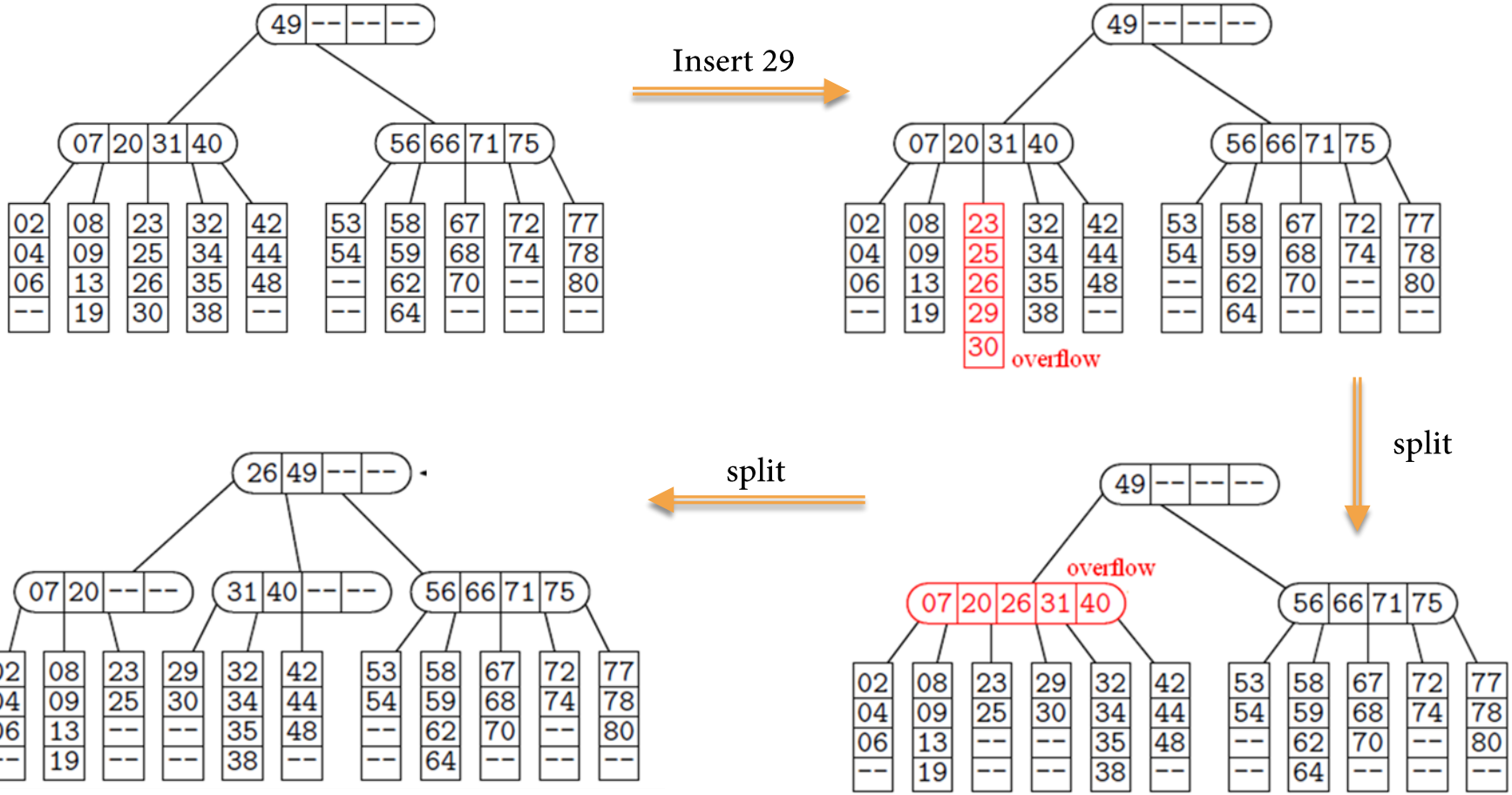
Overflow condition:
NumKeysInNode = 3



Insertion in B-Trees

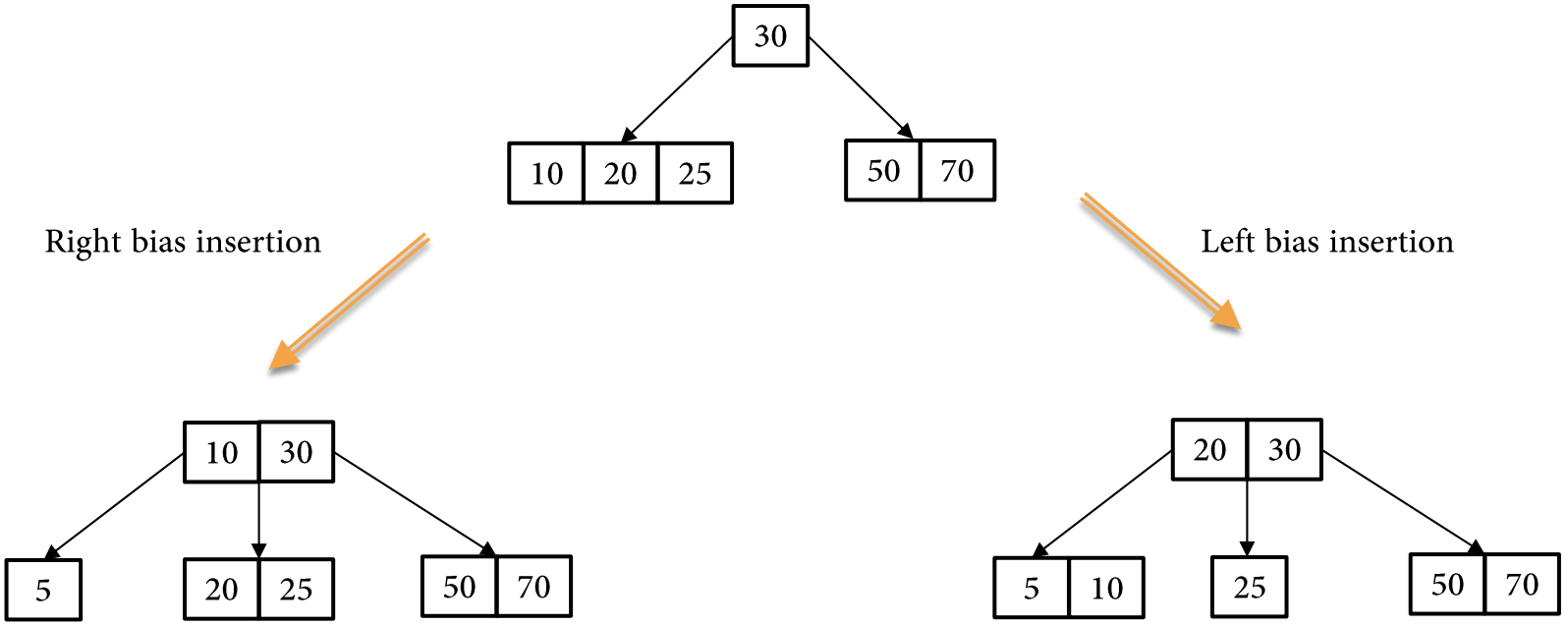
- Insertion in a B-tree of odd order
  - Insert 29 in a B-tree of order 5

Overflow condition:
NumKeysInNode = 5



Insertion in B-Trees

- Insertion in a B-tree of even order
  - At each node, the insertion can be done in two different ways:
  - right-bias: The node is split such that its right subtree has more keys than the left subtree.
  - left-bias: The node is split such that its left subtree has more keys than the right subtree.
  - Example: Insert the key 5 in the following order 4:



B-Tree Insertion Algorithm

```
insertKey(x):
    // 1. Duplicate check
    if (x already exists in the tree)
        throw exception
    // 2. Insert x into its leaf
    current = findLeaf(x)
    insert x into current in sorted order
    if (current is not overflow)
        return
    // 3. Handle overflows bottom-up
    do:
        // --- Split current node ---
        if (m is odd):
            split current into:
                ls = left m/2 keys
                rs = right m/2 keys
            w = middle key
        else:   // m is even
            choose one:
                right-bias  -> rs = m/2 keys,    ls = (m-1)/2 keys
                left-bias   -> rs = (m-1)/2 keys ls = m/2 keys
            w = middle key from split
        parent = current's parent

        // 4. If current is root ⟶ create new root
        if (parent == null):
            create newRoot with key w
            newRoot.leftChild  = ls
            newRoot.rightChild = rs
            tree.root = newRoot
            return

        // 5. Otherwise:
            insert promoted key into parent
            replace current with ls in parent's child list
            insert w and rs as new child in parent

        current = parent

    while (current overflows)

    return
```

Deletion in B-Tree

- Deletion algorithm:
  - If a node underflows, rotate the appropriate key from the adjacent right- or left-sibling if the sibling contains at least $\lceil m / 2 \rceil$ keys; otherwise perform a merging.

Node	Underflow: after deletion
Non-root	$\lceil m/2 \rceil - 2$ keys
root node does not underflow	

Min #keys:
$\lceil m/2 \rceil - 1$

- There are five deletion cases:

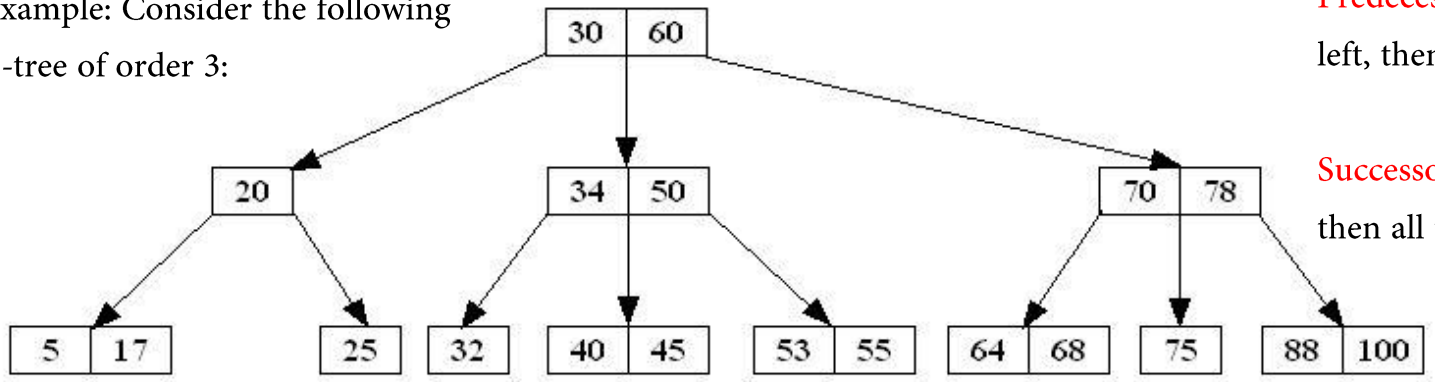
#	Case	Action
1	The leaf does not underflow	None
2	The leaf underflows and the adjacent right sibling has at least $\lceil m / 2 \rceil$ keys.	a left key-rotation
3	The leaf underflows and the adjacent left sibling has at least $\lceil m / 2 \rceil$ keys.	right key-rotation
4	The leaf underflows and each of the adjacent right/left sibling has at least $\lceil m / 2 \rceil$ key	left/right key-rotation
5	The leaf underflows and each adjacent sibling has $\lceil m / 2 \rceil$ - 1 keys	merging

- A key rotation must always be attempted before a merging

Deletion in B-Tree

- If the key to be deleted is not in a leaf, swap it with either its successor or predecessor (each will be in a leaf).
  - The successor of a key k is the smallest key greater than k. The smallest key to the right
  - The predecessor of a key k is the largest key smaller than k. The largest key to the left

Example: Consider the following B-tree of order 3:



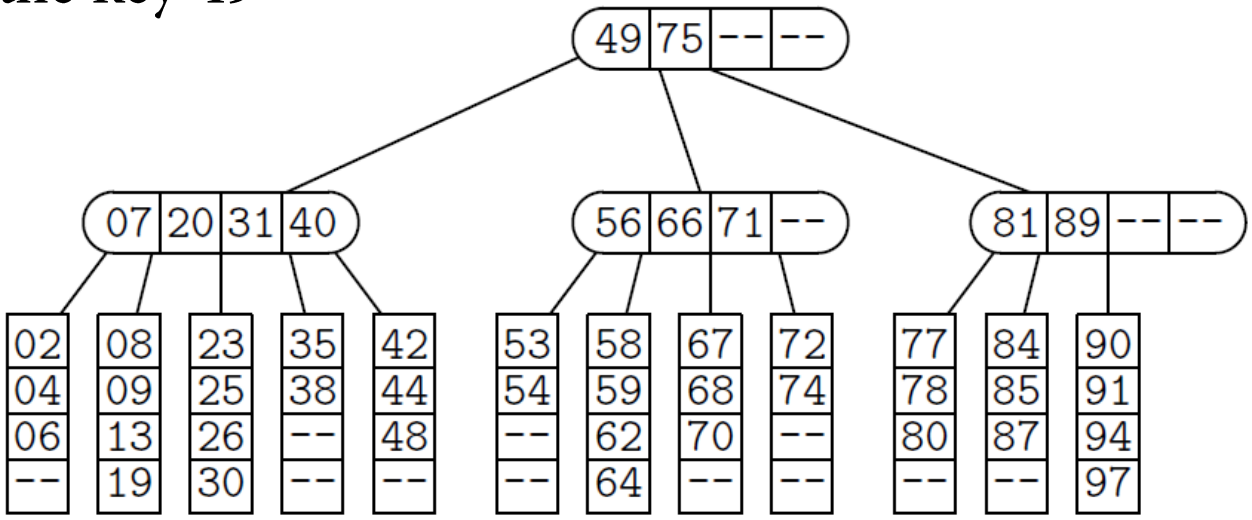
Predecessor: One step to the left, then all the way to the right

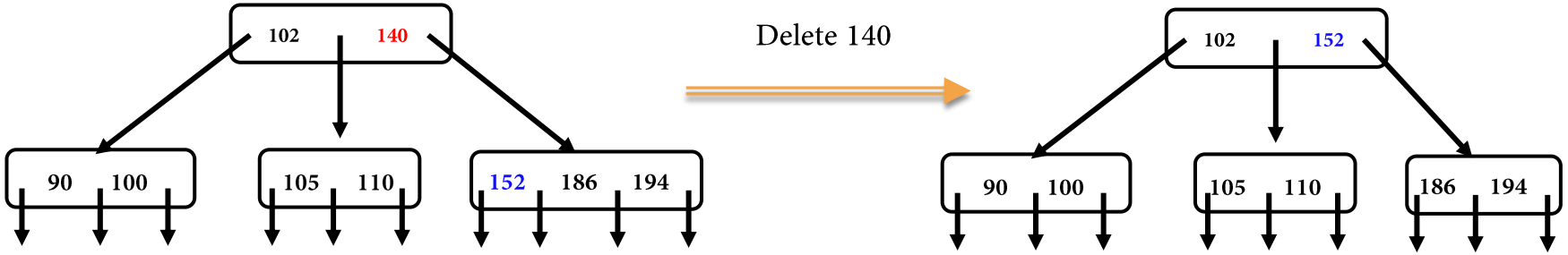
Successor: One step to the right, then all the way to the left

Non-leaf key	20	30	34	50	60	70	78
predecessor	17	25	32	45	55	68	75
successor	25	32	40	53	64	75	88

Deletion in B-Tree…

- In the following B-tree of order 5
- Delete the key 75 : Replace it with 77 or 74? 77
- Delete the key 49





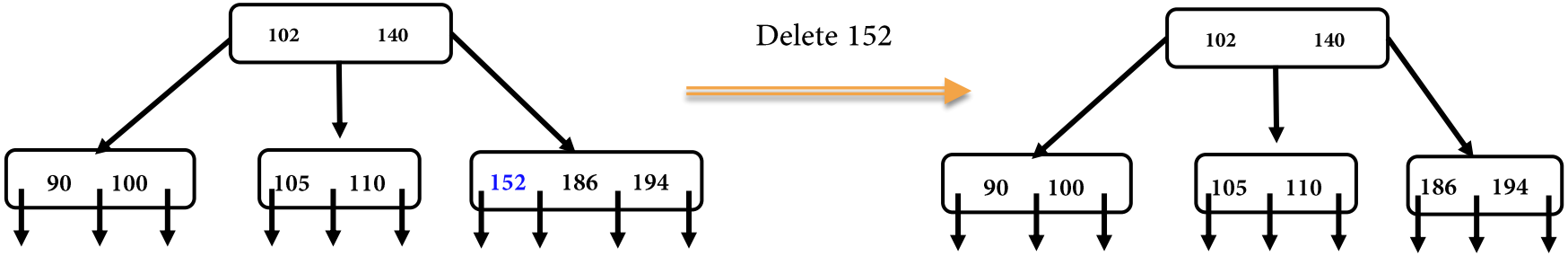
Deletion in B-Tree

- Case1: The leaf does not underflow.
- Example: **B-tree of order 4**

Underflow condition:

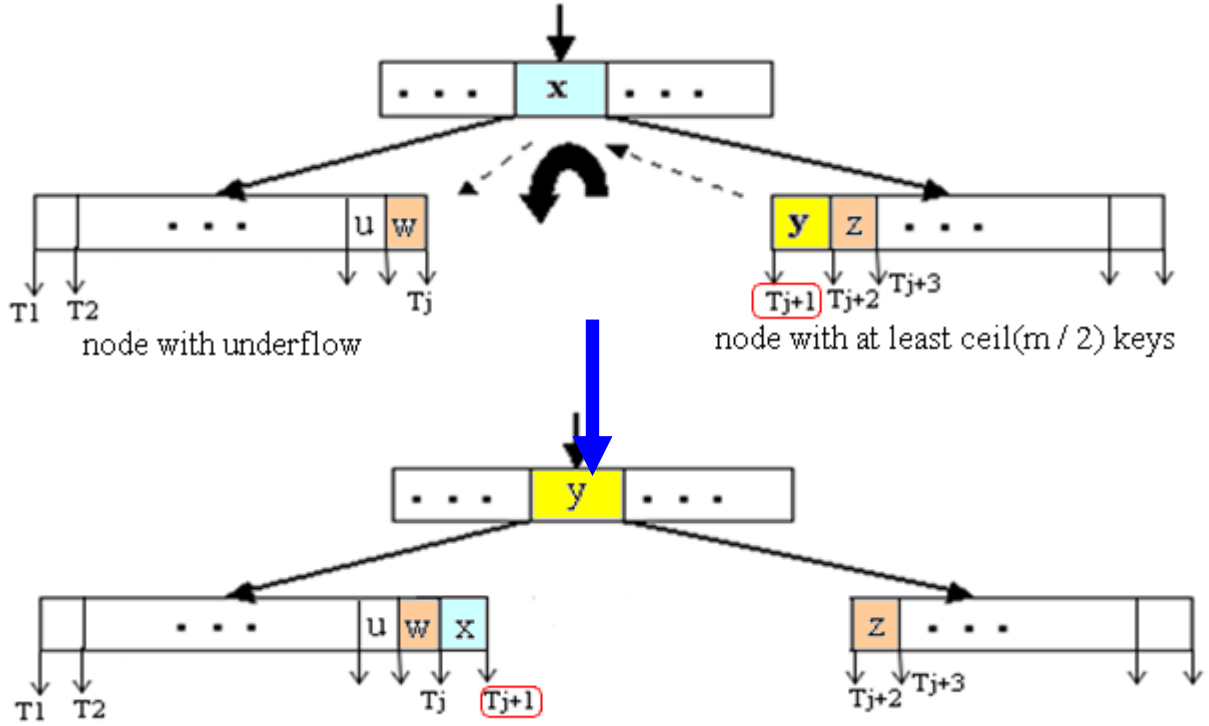
Ceil(4/2) – 2 = 0

Min #Keys = $\lceil m / 2 \rceil - 1 = \lceil 4 / 2 \rceil - 1 = 1$



Deletion in B-Trees (cont'd)

- Case2: The leaf underflows and the adjacent right sibling has at least $\lceil m / 2 \rceil$ keys.
- Perform a left key-rotation:
- 1. Move the parent key **x** that separates the siblings to the node with underflow.
- 2. Move **y**, the minimum key in the right sibling, to where the key **x** was.
- 3. Make the old left subtree T_{i+1} of **y** to be the new right subtree of **x**.



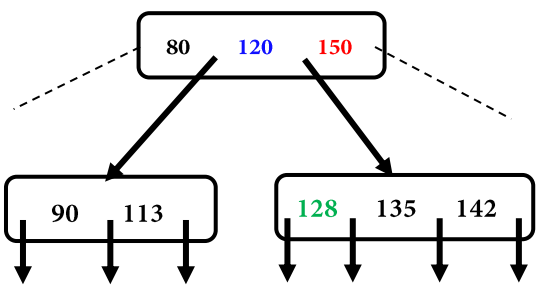
Deletion in B-Trees (cont'd)

- Example: **B-tree of order 5**

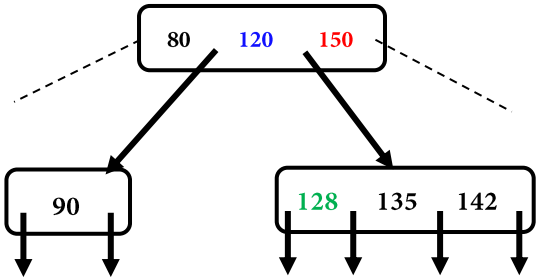
Min #Keys = ⌈ m / 2 ⌉ - 1 = ⌈ 5 / 2 ⌉ - 1 = 2

Underflow condition:

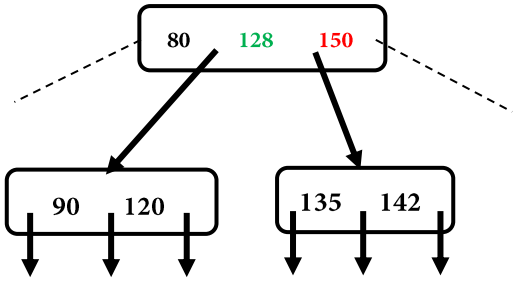
Ceil(5/2) – 2 = 1



Delete 113

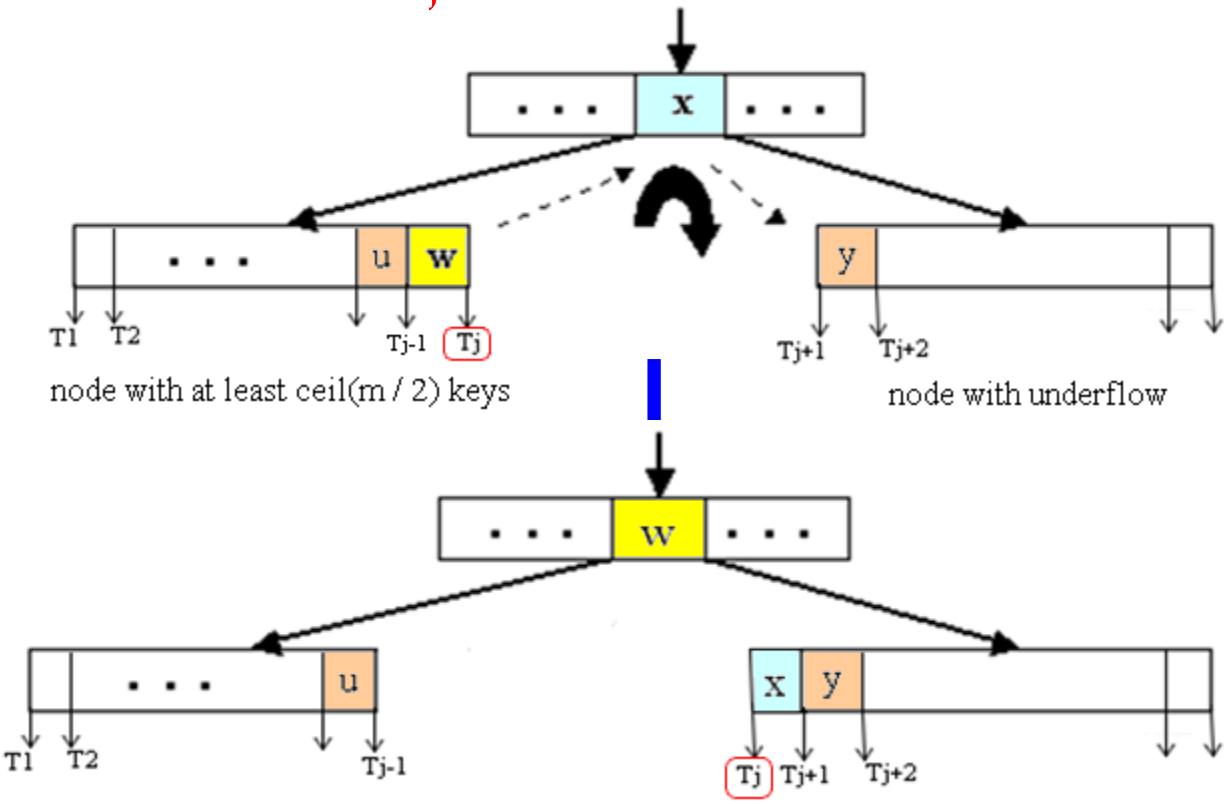


Underflow: Rotate 128 from adjacent right sibling



Deletion in B-Trees (cont'd)

- Case 3: The leaf underflows and the adjacent left sibling has at least $\lceil m / 2 \rceil$ keys.
- Perform a right key-rotation:
- 1. Move the parent key **x** that separates the siblings to the node with underflow.
- 2. Move **w**, the maximum key in the left sibling, to where the key **x** was.
- 3. Make the old right subtree T_j of **w** to be the new left subtree of **x.**



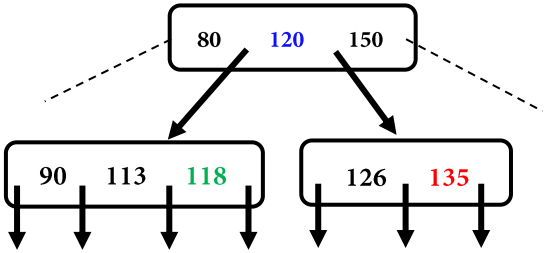
Deletion in B-Trees (cont'd)

- Example: **B-tree of order 5**

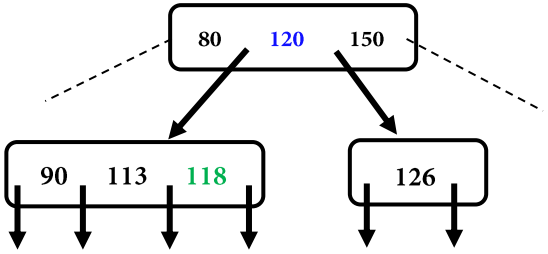
Min #Keys = ⌈ m / 2 ⌉ - 1 = ⌈ 5 / 2 ⌉ - 1 = 2

Underflow condition:

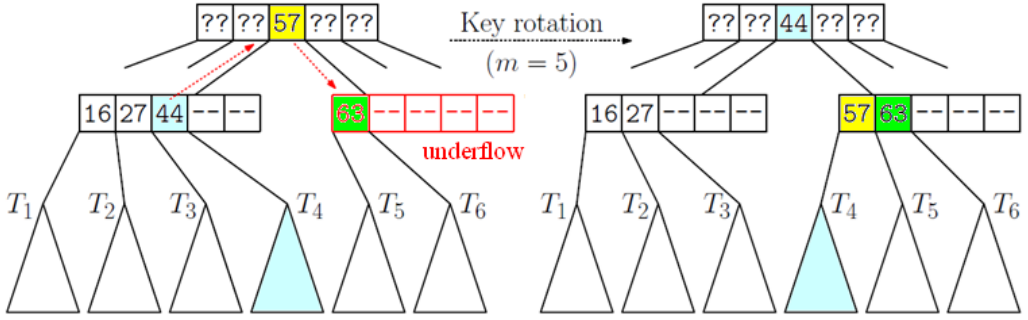
Ceil(5 / 2) – 2 = 1

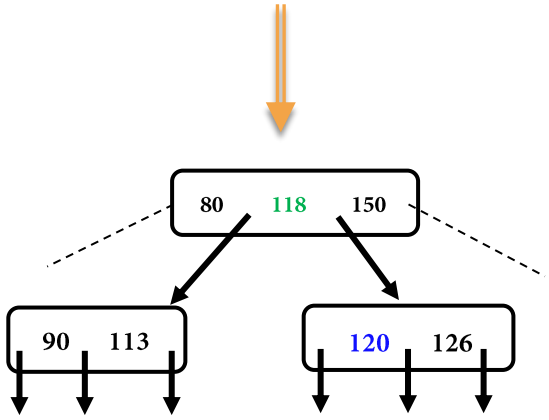


Delete 135



Underflow. Rotate 118 from adjacent left sibling





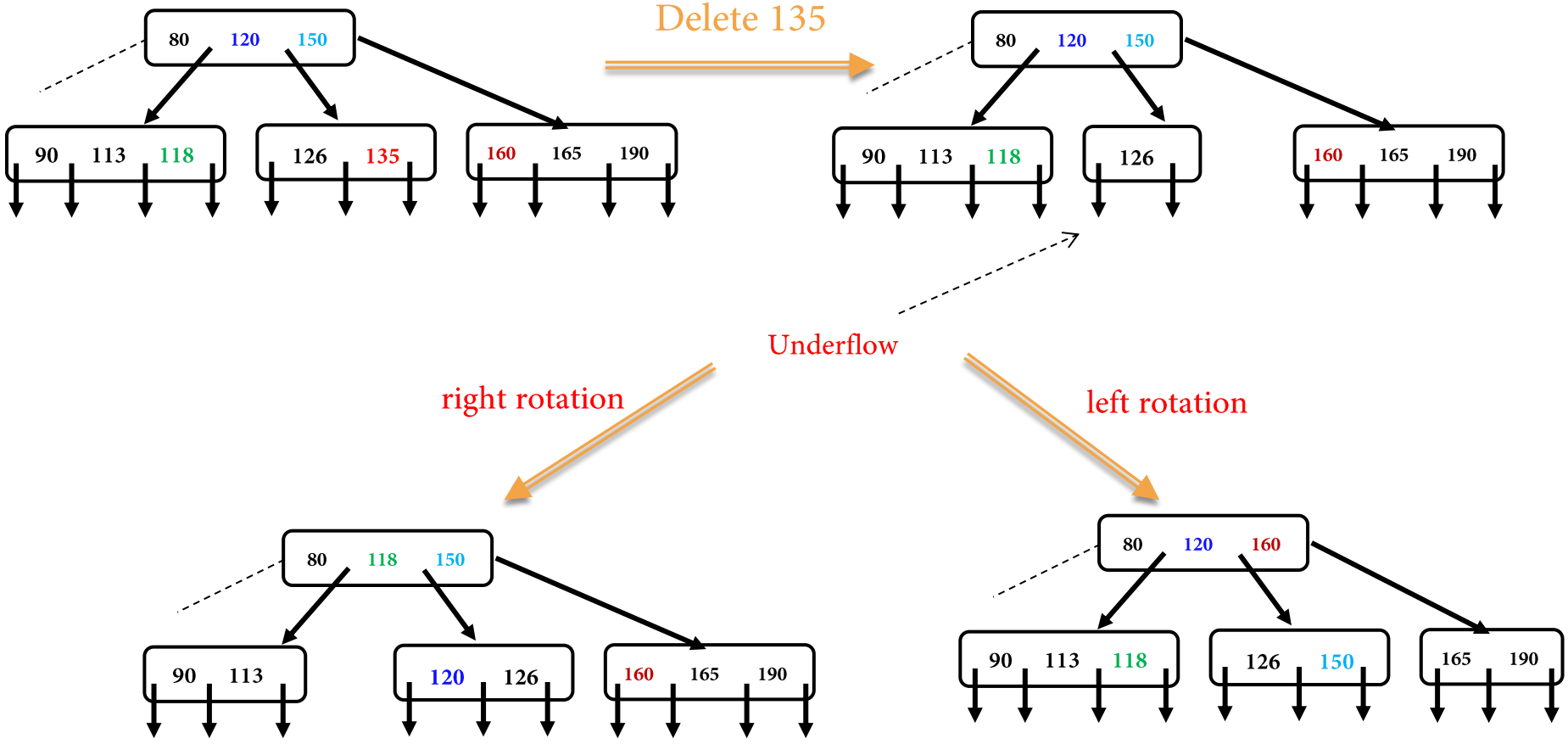
Deletion in B-Trees (cont'd)

- Case 4: The leaf underflows and each of the adjacent right sibling and the adjacent left sibling has at least $\lceil m / 2 \rceil$ keys.

Min #Keys = $\lceil m / 2 \rceil - 1 = \lceil 5 / 2 \rceil - 1 = 2$

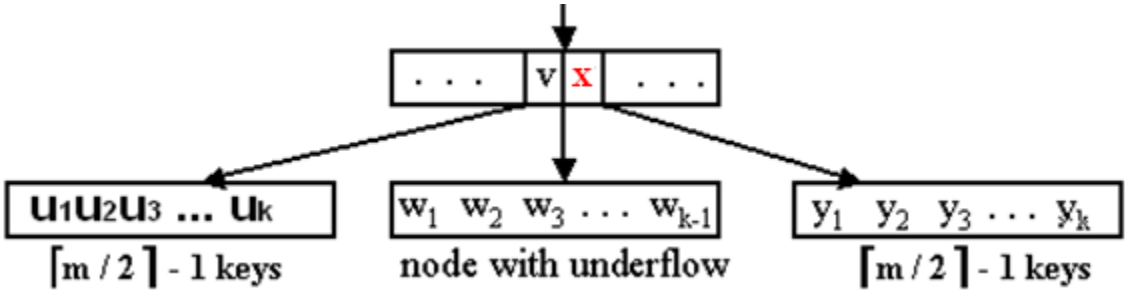
- Example: **B-tree of order 5**

Underflow condition:
Ceil(5 / 2) – 2 = 1

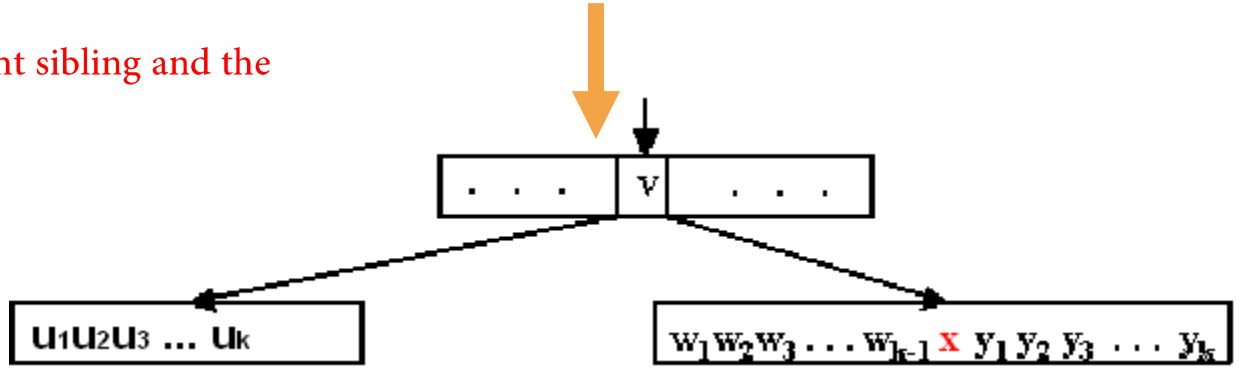


Deletion in B-Trees (cont'd)

- Case 5:The leaf underflows and each adjacent sibling has $\left\lceil \frac{m}{2} \right\rceil$ - 1 keys.
  - The merging could be done by using the right sibling.



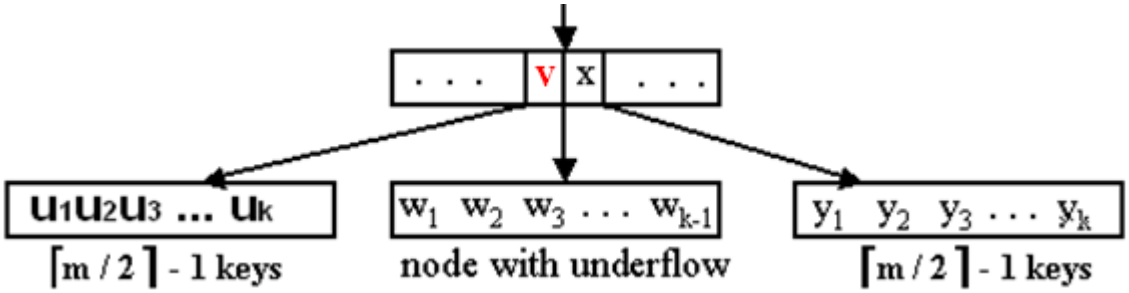
Merge node, right sibling and the separating key x



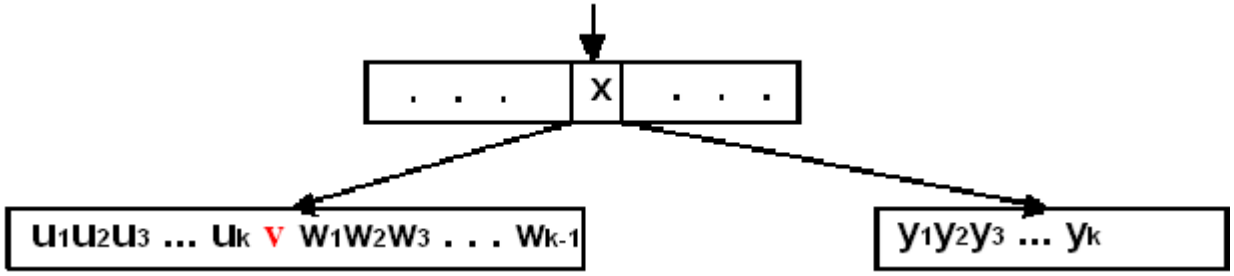
If the parent of the merged node underflows, the deletion algorithm propagates upward. In the limit, a root with one key is deleted and the height decreases by one.

Deletion in B-Trees (cont'd)

- The merging could also be done by using the left sibling instead of the right sibling.



merge node, left sibling and the separating key v



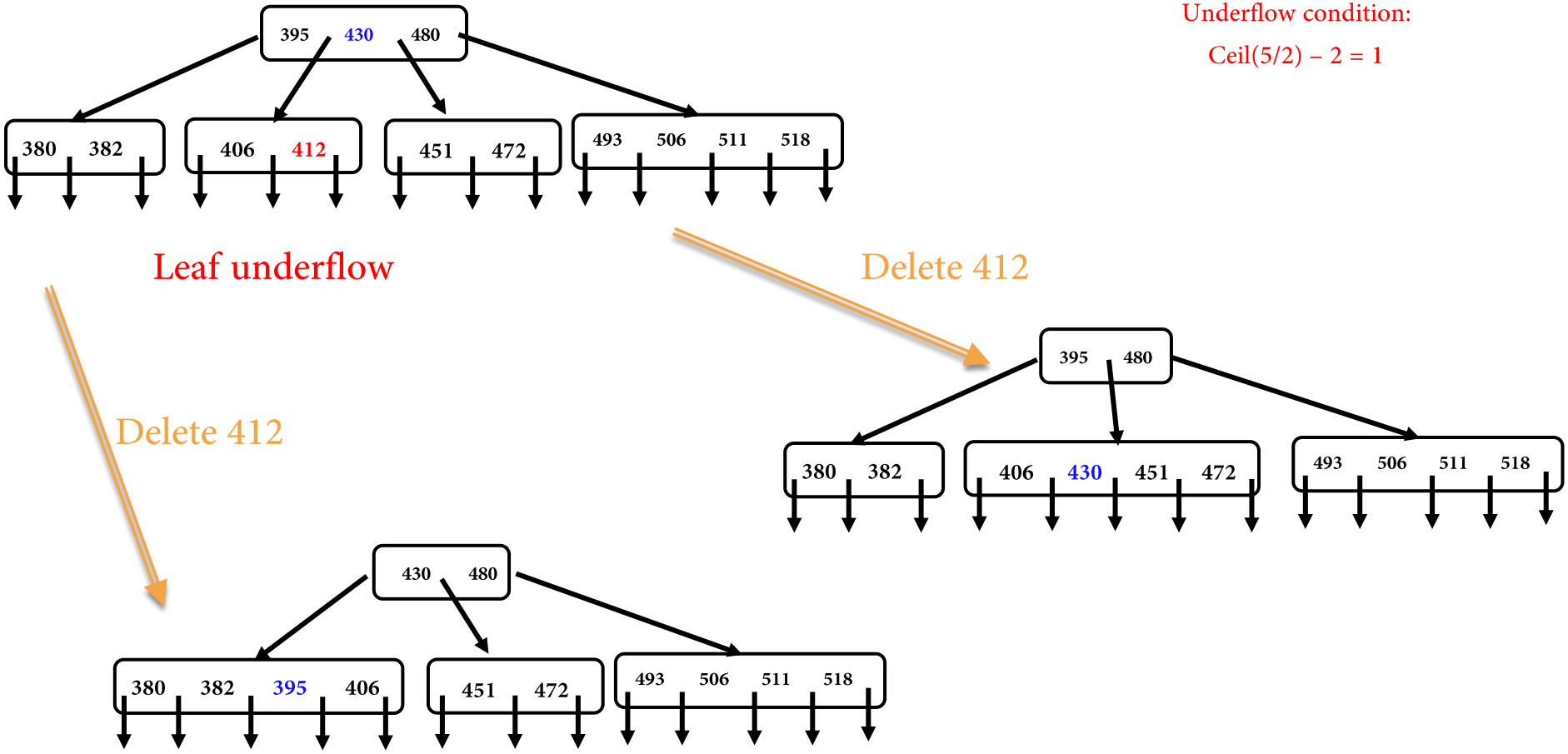
Deletion in B-Trees (cont'd)

- Example: A **B-tree of order 5**

Min #Keys = ⌈ m / 2 ⌉ - 1 = ⌈ 5 / 2 ⌉ - 1 = 2

Underflow condition:

Ceil(5/2) – 2 = 1



The parent of the merged node does not underflow. The deletion algorithm does not propagate upward.

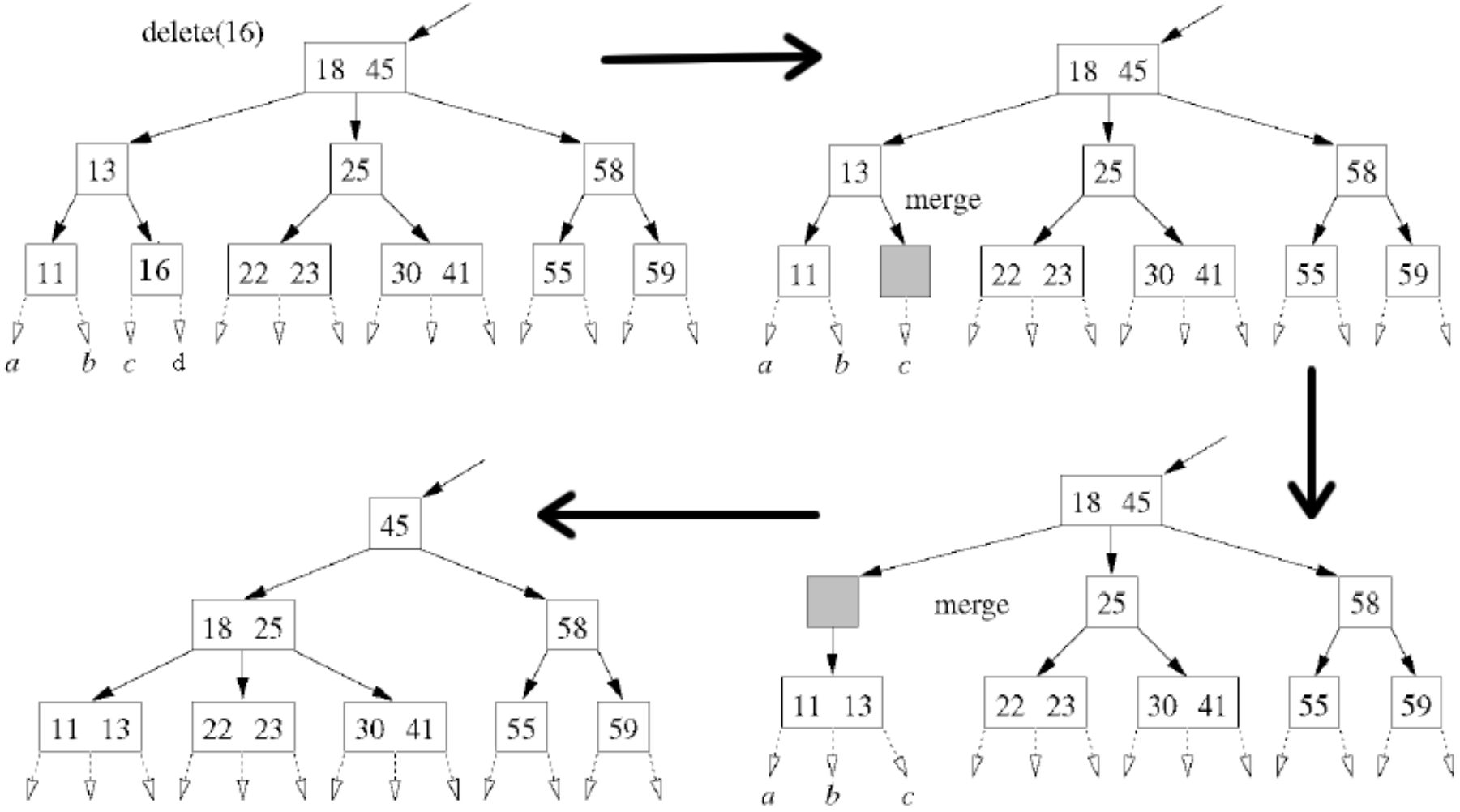
Deletion in B-Trees (cont'd)

- Example: Deleting 16 in a B-tree of order 3

Min #Keys = ⌈ m / 2 ⌉ - 1 = ⌈ 3 / 2 ⌉ - 1 = 1

Underflow condition:

Ceil(3/2) – 2 = 0



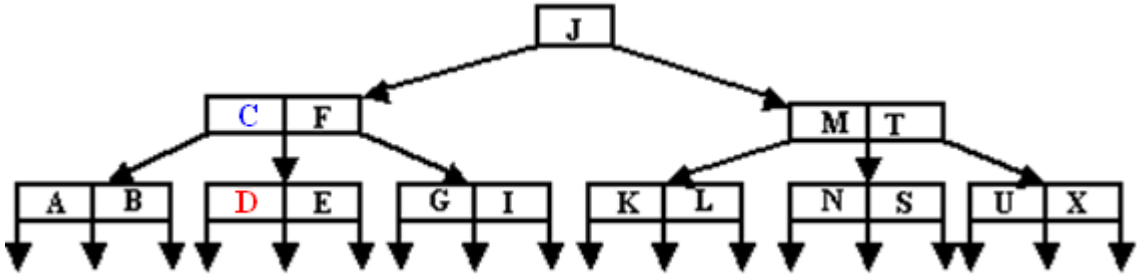
Deletion in B-Trees (cont'd)

- Example: **B-tree of order 5**

Min #Keys = ⌈ m / 2 ⌉ - 1 = ⌈ 5 / 2 ⌉ - 1 = 2

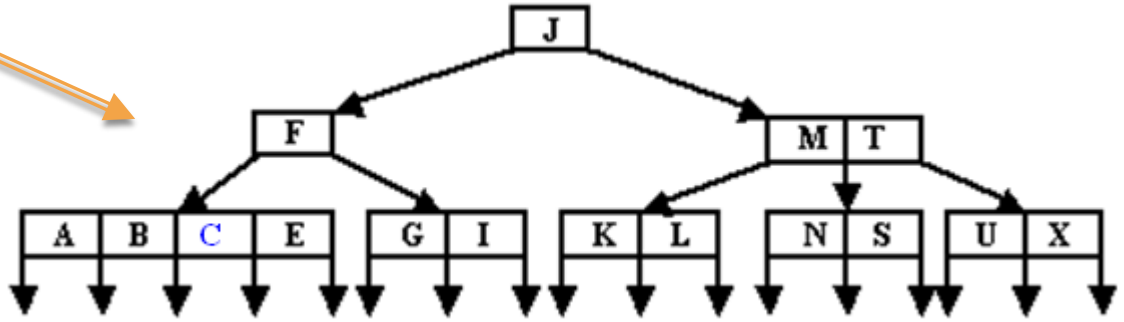
Underflow condition:

Ceil(5/2) – 2 = 1

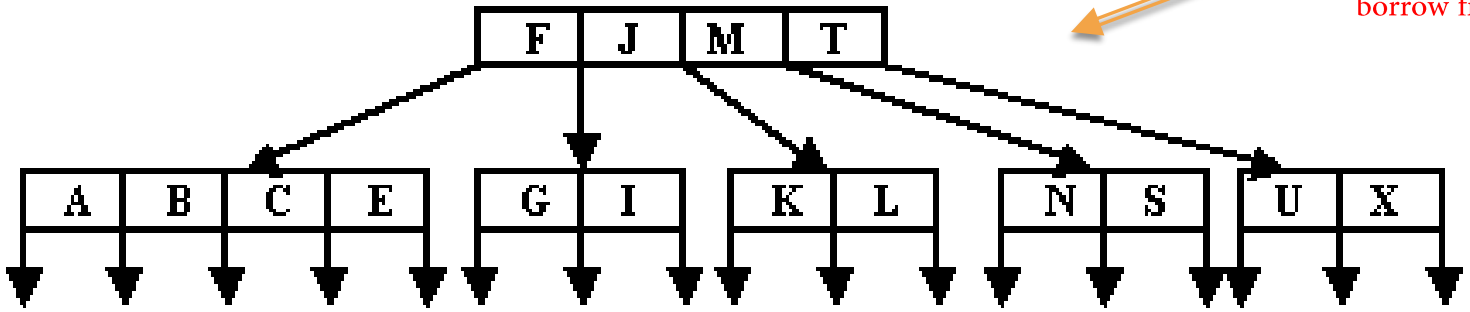


Delete D

leaf underflow. Cannot borrow from adjacent siblings => merge

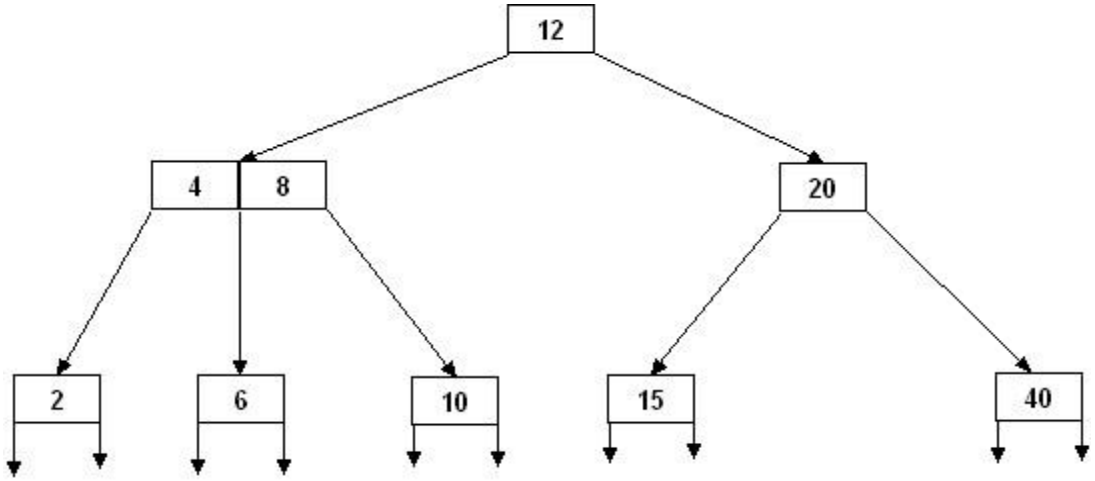


Internal node underflow. Cannot borrow from adjacent sibling => merge



Deletion in B-Trees (cont'd)

- Example: Delete the key 40 in the following B-tree of order 3:

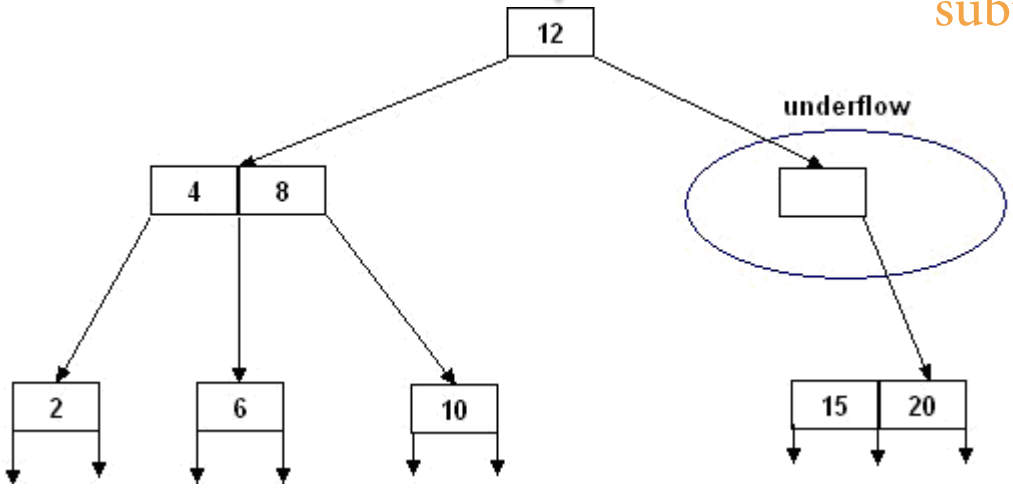


Min #Keys = $\lceil m / 2 \rceil - 1 = \lceil 3 / 2 \rceil - 1 = 1$

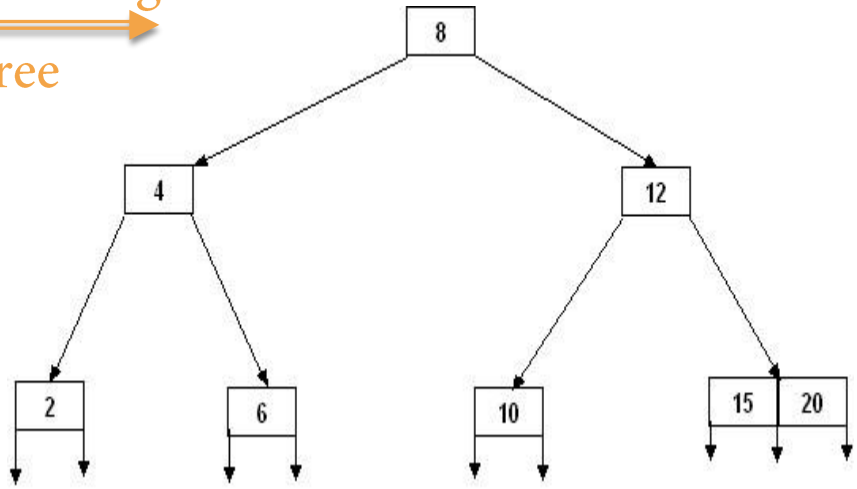
Underflow condition:

Ceil(3/2) – 2 = 0

merge 15 and 20



rotate 8 and its right subtree



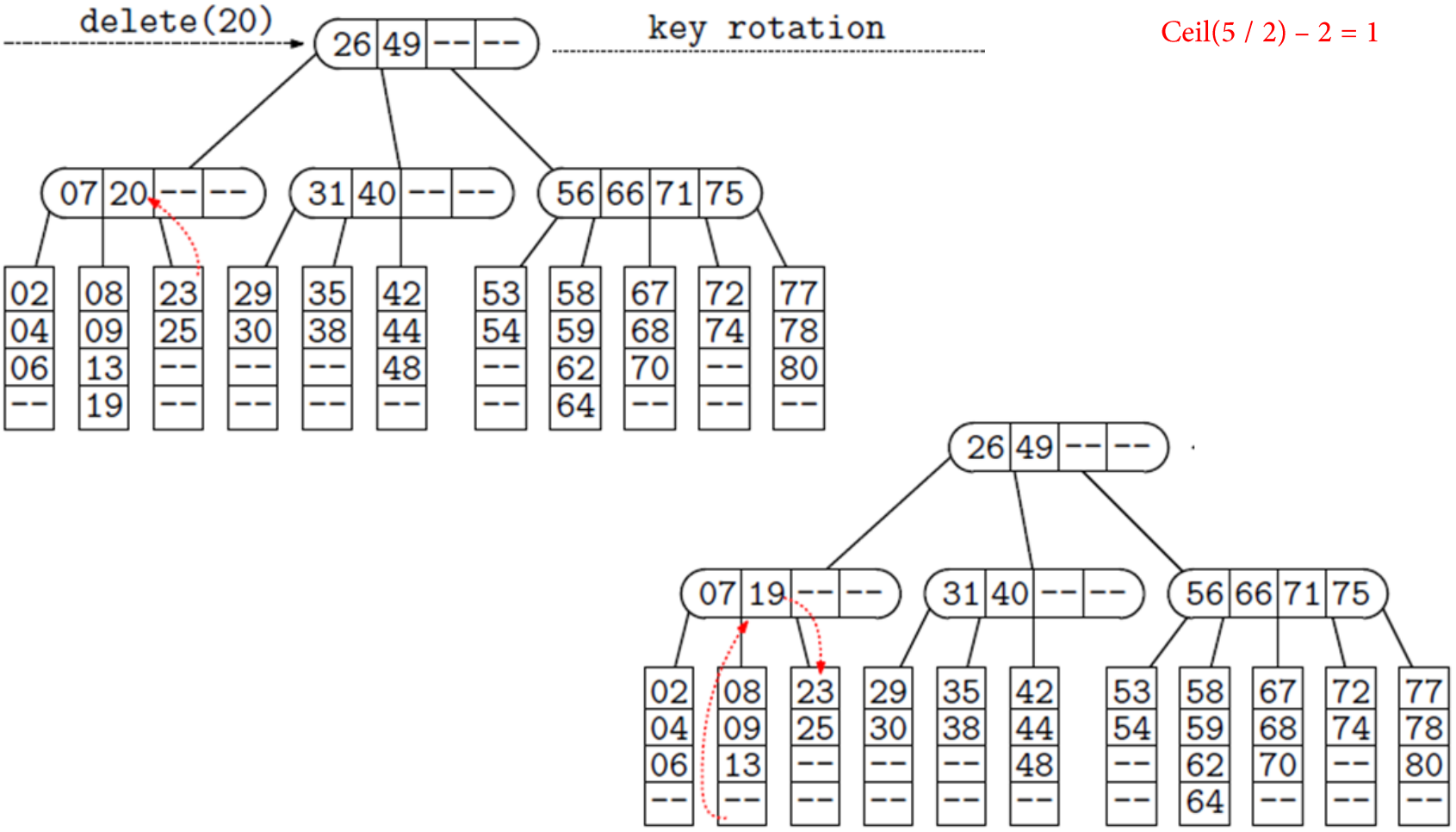
More Examples on Deletion (Self Study)…

- Example: Deleting 20 in a B-tree of order 5

Min #Keys = ⌈ m / 2 ⌉ - 1 = 1 = ⌈ 5 / 2 ⌉ - 1 = 2

Underflow condition:

Ceil(5 / 2) – 2 = 1



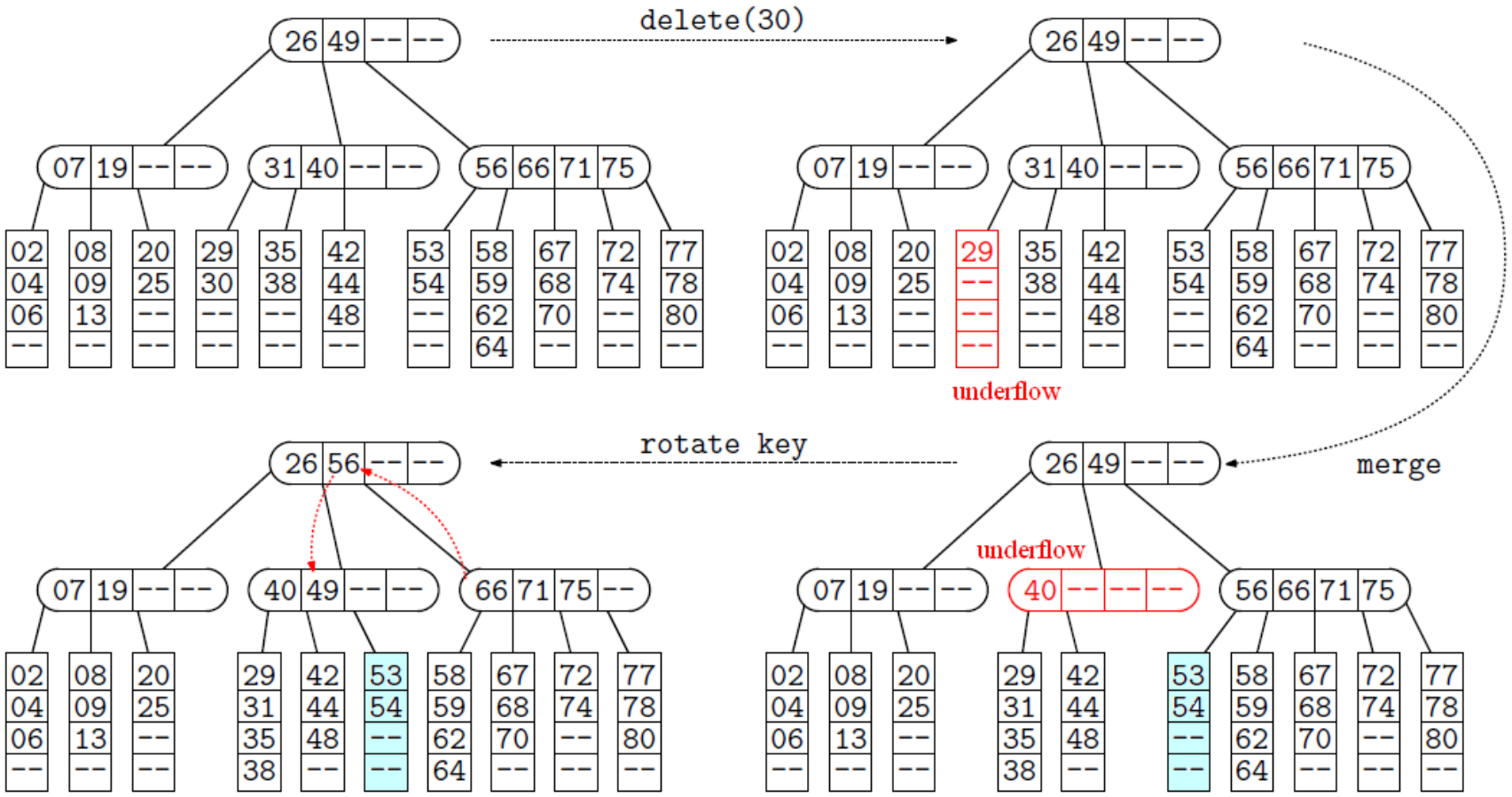
More Examples on Deletion (Self Study)

- Example: Delete the key 30 in the following B-tree of order 5:

Min #Keys = ⌈ m / 2 ⌉ - 1 = 1 = ⌈ 5 / 2 ⌉ - 1 = 2

Underflow condition:

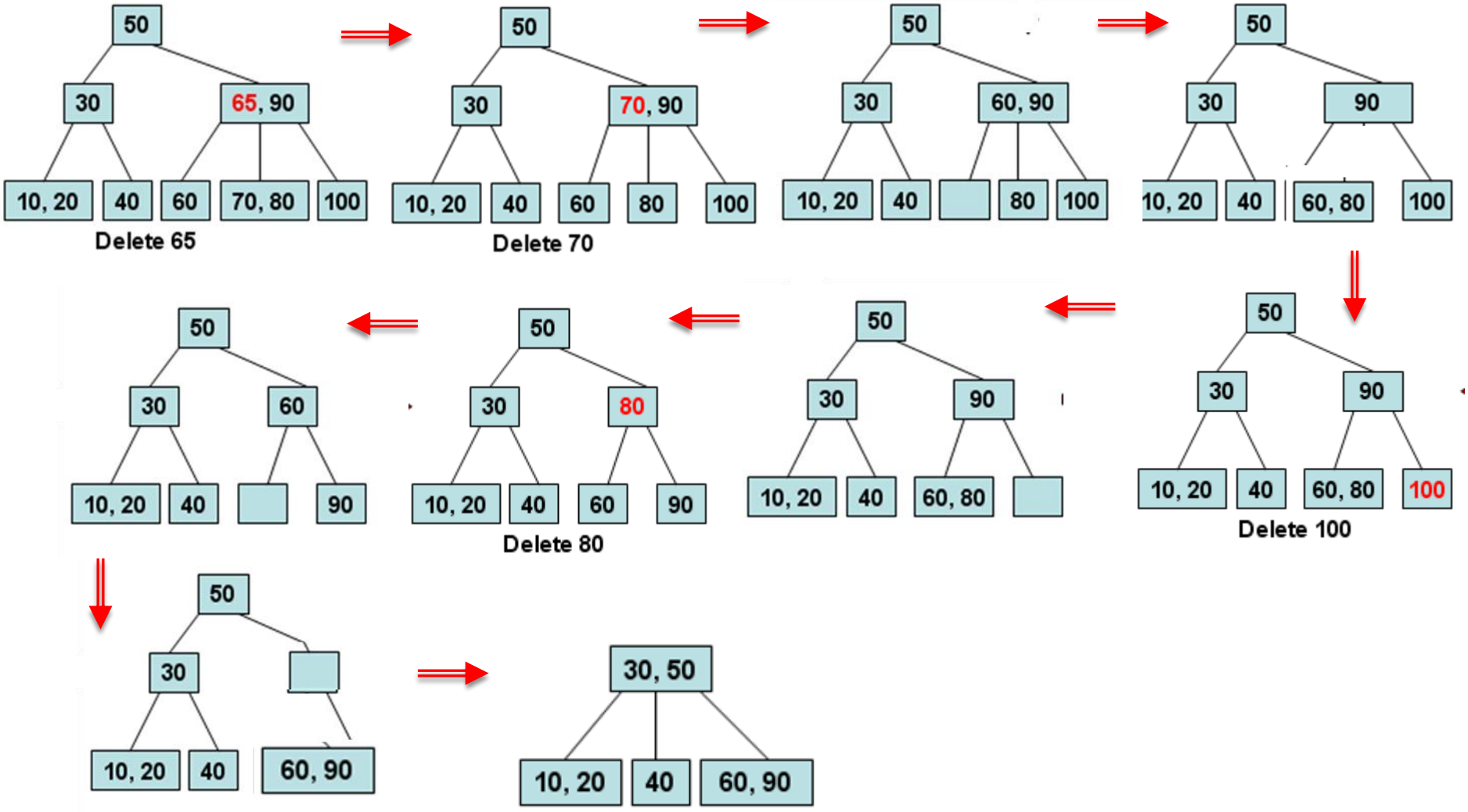
Ceil(5/2) – 2 = 1



More Examples on Deletion (Self Study)…

Underflow condition:
Ceil(3 / 2) – 2 = 0

- Example: Delete 65, 70, 100, and 80 in this order in the following B-tree of order 3



B-Tree Deletion Algorithm

```
deleteKey(x):

    if x not found: error

    // Ensure deletion happens in a leaf
    if x is in internal node:
        replace x with successor/predecessor

    remove x from the leaf
    current = leaf

    // If no underflow, stop
    if current has ≥ ceil(m/2) − 1 keys:
        return

    // Fix underflow bottom-up
    while current has < ceil(m/2) − 1 keys:

        sibling = adjacent sibling of current

        // Rotation
        if sibling has ≥ ceil(m/2) keys:
            rotate a key between current, parent, and
sibling
            break

        // Merge
        merge current, sibling, and the parent key
        parent = current's parent

        if parent is root and empty:
            make merged node the root
            return

        current = parent      // continue upward

    return
```